Sistema de Gestión Documental

Realizado por:
Dayana Sofía Orozco Mendóza

Manual Técnico

ÍNDICE

03 Descripción general

04 Requisitos previos

05 Instalación y puesta en marcha

11 Estructura del proyecto

12 Arquitectura del servidor

15 Referencia de la API

20 Arquitectura del cliente

21 Reglas de negocio importantes

22 Solución de problemas

23 Dependencias

SISTEMA DE GESTIÓN DOCUMENTAL

Sistema de Gestión Documental Arquitectura: cliente-servidor (Angular 21 + Node.js/Express + MongoDB)

1. DESCRIPCIÓN GENERAL

El sistema permite almacenar, organizar, consultar y compartir archivos .txt y .md entre usuarios de una organización. Consta de dos aplicaciones independientes:

Código fuente:
https://github.com/sofiaOroM/GestorDocumental.git

Diagramas: Sistema de Gestión Documental

Componente	Tecnología	Responsabilidad
Cliente	Angular 21 (componentes standalone, signals)	Interfaz de usuario; consume la API REST
Servidor	Node.js + Express 5	Lógica de negocio, seguridad y único acceso a la base de datos
Base de datos	MongoDB (+ GridFS)	Metadatos, relaciones, historial y contenido de los archivos

2. REQUISITOS PREVIOS

Software	Versión	Nota
Node.js	20.19 o superior (o 22.12+)	Mínimo que exige Angular 21
npm	el que acompaña a Node.js	
MongoDB	4.4 o superior (se recomienda 6.0+)	Instancia local en localhost:27017
Navegador	Chrome, Edge o Firefox actuales	
Internet	Opcional	Solo para la fuente de iconos Material Icons (ver sección 9)

No se necesita replica set ni configuración especial de MongoDB.

3. INSTALACIÓN Y PUESTA EN MARCHA

3.1 Servidor

bash

- cd backend/servidor
- npm install
- cp .env.example .env # en Windows (cmd): copy .env.example .env

Verifica que MongoDB esté en ejecución y carga los datos de demostración:

bash

- npm run seed # Borra la base y la recrea con datos de prueba
- npm run dev # servidor con recarga automática (o: npm start)

La API queda en http://localhost:3000/api.

3.2 Cliente

bash

- cd frontend
- npm install
- npm start # http://localhost:4200npm install

Si el servidor corre en otra dirección o puerto, cambia API_URL en src/app/core/config.ts.

3.3 Variables de entorno (.env del servidor)

Variable	Valor por defecto	Descripción
PORT	3000	Puerto de la API
MONGO_URI	mongodb://localhost:27017	Cadena de conexión a MongoDB
DB_NAME	gestion_documental	Nombre de la base de datos
JWT_SECRET	*(sin definir)*	Secreto para firmar los tokens. Con NODE_ENV=production es obligatorio (el servidor no arranca sin él); en desarrollo, si falta, se usa un secreto de prueba y se muestra un aviso
CORS_ORIGIN	´http://localhost:4200	Origen del cliente permitido por CORS
DEFAULT_QUOTA_MB	100	Espacio total asignado por defecto a cada usuario nuevo (MB)
MAX_FILE_MB	*(sin definir)*	Opcional. Si se define, limita el tamaño por archivo en MB; si no, no hay límite

El archivo .env no se sube al repositorio (está en .gitignore); sí se incluye .env.example.

4. ESTRUCTURA DEL PROYECTO

4.1 Backend

```
backend/servidor/
├── scripts/seed.js Carga inicial de MongoDB
└── src/
    ├── server.js Arranque: conecta a MongoDB, crea índices, escucha
    ├── app.js Express: CORS, JSON, montaje de rutas, manejo de errores
    ├── config.js Lectura de variables de entorno
    ├── db.js Conexión, bucket GridFS y creación de índices
    ├── middleware/
    │   ├── auth.js authenticate (JWT) y requireAdmin
    │   └── errors.js Respuestas de error en JSON
    ├── routes/ auth, users, folders, files, shares, trash, activity, reports
    └── utils/
        ├── access.js Niveles de acceso y herencia de permisos
        ├── tree.js Jerarquía de carpetas (ancestors, mover, nombres únicos)
        ├── gridfs.js Guardar y borrar contenido en GridFS
        ├── activity.js Registro del historial
        ├── sharesQuery.js Pipeline compartido de comparticiones
        └── http.js Errores, validaciones, fechas y paginación
```

4.2 Frontend

```
frontend/src/app/
├── core/ Api, Auth, interceptor JWT, guards, modelos, utilidades
├── dialogs/ Mover, compartir, visor/editor, historial, nombre
└── pages/ login, shell, explorer, shared, trash, activity, reports, users
```

5. ARQUITECTURA DEL SERVIDOR

5.1 Flujo de una petición

Petición HTTP
→ CORS y JSON (app.js)
→ authenticate valida el JWT y recarga al usuario (rechaza inactivos)
→ ruta valida datos de entrada
→ loadResource carga el recurso y calcula el nivel de acceso (access.js)
→ operación en MongoDB
→ logActivity registra la operación en activity_log
→ respuesta JSON (los errores pasan a errorHandler)

Express 5 envía automáticamente al manejador de errores cualquier excepción de una función async, por lo que las rutas no necesitan try/catch.

5.2 Seguridad

- **Autenticación:** JWT firmado con JWT_SECRET, válido 8 horas. El cliente lo envía en la cabecera Authorization: Bearer <token>.
- **Contraseñas:** se guardan como hash bcrypt (bcryptjs, 10 rondas).
- **Usuarios inactivos:** authenticate consulta al usuario en cada petición, por lo que un usuario desactivado pierde el acceso de inmediato aunque conserve un token vigente.
- **Autorización:** la verifica siempre el servidor. Ocultar botones en el cliente es solo comodidad de interfaz.
- **Validaciones:** extensión .txt/.md, nombres sin caracteres \ / : * ? " < > | (máximo 120), identificadores con formato de ObjectId.

6. REFERENCIA DE LA API

Base: http://localhost:3000/api. Salvo el login, todas las rutas requieren el token JWT.

Autenticación y usuarios

Método	Ruta	Descripción	Acceso
POST	/auth/login	Inicia sesión { email, password } → { token, user }	Público
GET	/auth/me	Datos del usuario actual (incluye almacenamiento usado)	Autenticado
GET	/users/directory	Usuarios activos, para elegir con quién compartir	Autenticado
GET	/users	Lista de usuarios	Admin
POST	/users	Crea usuario { nombre, email, password, rol, cuotaMB } (cuotaMB opcional)	Admin
PATCH	/users/:id	Cambia nombre, rol, activo, contraseña o cuota { cuotaMB } (no puede ser menor al espacio ya usado)	Admin